

LC 3217: Delete Nodes From Linked List Present in Array

AMAZON

GOOGLE

Problem Statement

We are given an array and a linked list (LL). Any value present in the array must not be in the LL. Return the modified LL.

Key Concepts

- **Concept 1:** We need to store all numbers from the array in a **HashMap / HashSet** for $O(1)$ lookups.
- **Concept 2:** We need a **Dummy Node** so that we can keep track of the previous node. This needs to be done such that whenever the current node lands on a value which is in the array, then we can safely reestablish our links and delete the target node.

Example Walkthrough

Input LL: 1 → 2 → 3 → 4 → 5

nums = [2, 3]

Output LL: 1 → 4 → 5

Approaches

Approach 1: Linear Search (Brute Force)

For every node in the linked list, check if its value exists in the array by iterating through the array.

Time Complexity: $O(n \times m)$

Space Complexity: $O(1)$

Approach 2: Optimal Approach

Use a HashSet to store array elements for efficient lookup. Use a dummy node to keep track of the previous node, allowing us to easily bypass and drop nodes that need to be deleted.

Time Complexity: $O(n + m)$

Space Complexity: $O(m)$


```

/**
 * Definition for singly-linked list.
 * struct ListNode {
 *   int val;
 *   ListNode *next;
 *   ListNode() : val(0), next(nullptr) {}
 *   ListNode(int x) : val(x), next(nullptr) {}
 *   ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* modifiedList(vector<int>& nums, ListNode* head) {

        unordered_set<int> st;
        for(int n : nums) {
            st.insert(n);
        }

        ListNode* dummy = new ListNode(0);
        dummy->next = head;
        ListNode* prev = dummy;
        ListNode* curr = head;

        while(curr) {
            int value = curr->val;

            if(st.count(value)) {
                // Skip the current node
                prev->next = curr->next;
                curr = curr->next;
                continue;
            }

            // Move both pointers forward safely
            prev = curr;
            curr = curr->next;
        }

        return dummy->next;
    }
};

```

```
}  
};
```